Домашнее задание № 4
«Обеспечение отказоустойчивости средствами Apache ZooKeeper»

Ансамбль ZooKeeper и запасной координатор MapReduce

Выборы лидера, чекпоинт, эпоха и защита записи результата
Расчёт пар навыков PHP-вакансий из ДЗ 3

Выполнил: **Тищенко Павел Валерьевич**
Группа ИИИ-3-ИнжДан-1-1 (J3310), № 16
Репозиторий: `https://github.com/paulhowever/vk_data_eng_2026_ai_67_tischenko`

2026

1 Стенд и конфигурация

Таблица 1. Стенд

Машина	Apple M5, 10 ядер, 16 ГБ RAM, macOS; Docker Engine 29.2.0; Python 3.13, uv
Java	локально OpenJDK 17.0.19 (Homebrew), `JAVA_HOME=/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home`; в образе Temurin 17.0.20.1, `JAVA_HOME=/opt/java/openjdk`
ZooKeeper	3.9.6, официальный образ `zookeeper:3.9.6`, три контейнера `hh-zk1..3` (`zookeeper/compose.yaml`)
Клиент	kazoo 2.11.0, строка подключения `127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183`, тайм-аут сессии 10 с
Данные	RustFS из ДЗ 3 (`localhost:19000`), объект `raw/it_vacancies_2024.csv`, эталон `results/mapreduce/k2.csv`

Версии взяты из `hw4/outputs/stage1_java.txt`. Вместо трёх распакованных копий дистрибутива я поднял три сервера в Docker. Каждому из репозитория монтируется свой `zoo.cfg` и свой `myid`. Строки `server.N` у всех одинаковые, а `clientPort` и `dataDir` свои. Контейнер выполняет штатную команду `zkServer.sh start-foreground`, PID 1 в нём — `QuorumPeerMain /conf/zoo.cfg` (`stage2_roles.txt`).

```
$ cat zookeeper/zk2/conf/zoo.cfg                  # zk1, zk3 отличаются только dataDir и clientPort
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/zk/zk2/data
clientPort=2182
quorumListenOnAllIPs=true
admin.enableServer=false
4lw.commands.whitelist=srvr,stat,mntr,ruok,cons
server.1=zk1:2888:3888
server.2=zk2:2889:3889
server.3=zk3:2890:3890
```

Три строки в конфиге добавлены сверх минимума. С параметром `quorumListenOnAllIPs=true` сервер слушает порты кворума и выборов на всех интерфейсах контейнера, а не только на адресе, в который резолвится его собственное имя из `server.N`. Параметр `admin.enableServer=false` выключает встроенный HTTP AdminServer: он мне не нужен и по умолчанию занимает порт 8080. Через `4lw.commands.whitelist` я разрешил четырёхбуквенные команды, которыми смотрю состояние серверов: `srvr`, `stat`, `mntr`, `ruok` и `cons`. Порты кворума и выборов у серверов разные (2888/3888, 2889/3889, 2890/3890), хотя у каждого контейнера свой IP и хватило бы одной пары. Так конфиги совпадают с тем, что пришлось бы написать для трёх копий на одной машине.

2 Ансамбль из трёх серверов

Все шаги выполняет `zookeeper/stage2.sh`, выводы лежат в `hw4/outputs/stage2_*.txt`. Я запускал его сразу после `docker compose up -d`, поэтому сначала скрипт ждёт, пока все три сервера начнут отвечать `Mode: leader` или `Mode: follower`. Без этого первые выборы ещё идут, лидер не найден, и остановка «лидера» превращается в `docker compose stop` с пустым именем.

- **Роли.** Лидер — zk3, а zk1 и zk2 — последователи, это видно по `zkServer.sh status`. Команда `srvr` показывает у последователей `Zxid: 0xb000000ea`, а у лидера `0xc00000000`. Старшие 32 бита zxid — номер эпохи лидера. Новый лидер начал эпоху 0xc, но ни одной транзакции в ней ещё не было, а последователи показывают последнюю запись прошлой эпохи.
- **Минус один.** Остановил лидера zk3 командой `docker compose stop`. Скрипт опрашивает `zkServer.sh status` и увидел нового лидера zk2 не позже чем через 1,0 с после начала остановки. Точнее видно по журналу самого ZooKeeper (строки `LEADER ELECTION TOOK` на рис. 1): выборы заняли 214 мс у zk2, ставшего лидером, и 204 мс у zk1. Запись `create /minus1` через zk1 прошла и читается через zk2: два сервера из трёх — это большинство.

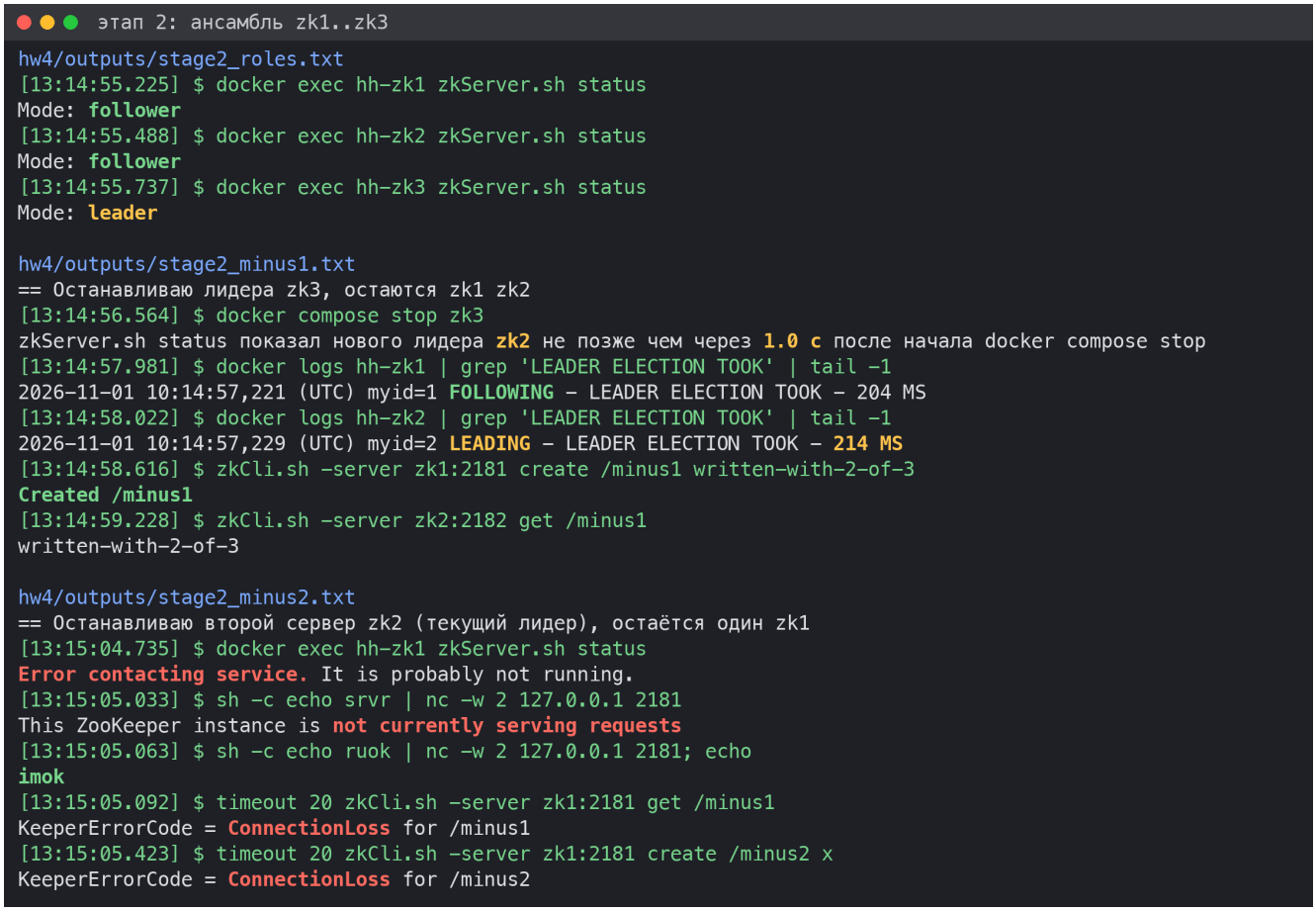


Рис. 1. Роли, остановка одного и двух серверов (фрагмент вывода `hw4/outputs/stage2_*.txt`, строки отобраны, команды и строки журнала ZK сокращены)

- **Минус два.** Остановил ещё и zk2. Процесс zk1 жив (`ruok` отвечает `imok`), но запросы он не обслуживает. На `srvr` сервер отвечает «This ZooKeeper instance is not currently serving requests», а `zkServer.sh status` пишет «Error contacting service». И чтение `get /minus1`, и запись `create /minus2 x` падают с `ConnectionLoss`, код выхода `zkCli` 1. Один сервер из трёх кворума не образует.
- **Возврат.** Запустил zk3 и zk2: все три снова работают (лидер zk2), и узел `/minus1`, записанный без zk3, читается и через zk3.

Почему ансамбль переизбрал лидера за доли секунды, а координатору в этапах 3–4 нужно около 12 с? Команда `docker compose stop` — мягкая остановка: Java-процесс получает SIGTERM, завершается и закрывает свои TCP-соединения. Последователи сразу видят, что связь с лидером пропала, и начинают выборы. Мой замороженный координатор ничего не закрывает, его соединения формально живы, и сервер ждёт, пока истечёт тайм-аут сессии. Лидером стал именно zk2, потому что при выборах голоса сравниваются по последнему zxid, а при равном zxid побеждает больший `myid`. У zk1 и zk2 zxid совпадал, и у zk2 номер больше. Выводы этапа собраны на рис. 1.

Эфемерный узел. «Окно 2» — это интерактивный `zkCli.sh` в отдельном контейнере `hh-zk-window2`, подключённый к zk2. Скрипт печатает каждую команду, которую отправляет в это окно, поэтому в выводе видно `окно 2> create -e /demo/eph "owned-by-window-2"`. Из «окна 1» `stat` показывает `ephemeralOwner = 0x2000000cc080001`, это сессия окна 2. Её согласованный тайм-аут я снял командой `cons` на zk2: `to=30000`, то есть 30 с (значение `zkCli` по умолчанию). Окно я «закрыл» командой `docker kill`: клиент умер, не закрыв сессию, как при закрытии терминала. Дальше скрипт раз в ≈0,12 с опрашивает `mntr` на zk1. Счётчик `zk_ephemerals_count` стал 0 не позже чем через 29,8 с после начала `docker kill`. Это чуть меньше 30 с, потому что тайм-аут отсчитывается от последнего пакета клиента, который был до `kill`, а сервер проверяет истечение по границам `tickTime` = 2 с. После этого `ls /demo` возвращает `[]`, а `stat /demo/eph` — `Node does`

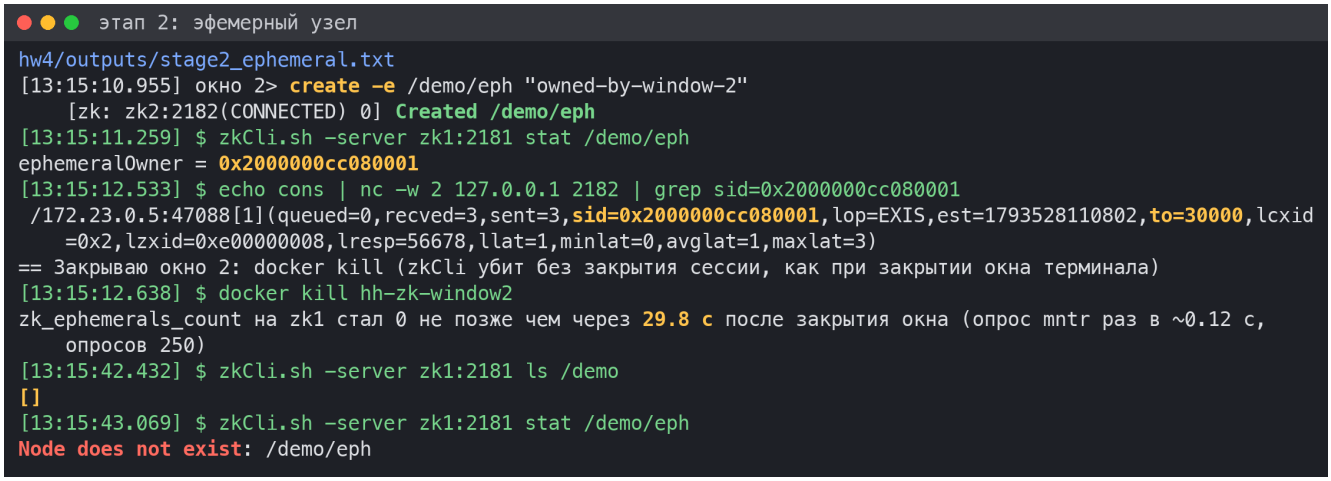


Рис. 2. Эфемерный узел исчезает после смерти сессии-владельца (фрагмент вывода `hw4/outputs/stage2_ephemeral.txt`, строки отобраны)

`not exist` (рис. 2).

3 Координатор с запасным

Пакет `hhlake.zk` работает поверх движка ДЗ 3: нарезка `split.plan` (26 чанков по 32 МиБ), map-функция `engine.run_map`, shuffle в S3, `jobs.merge_blob` и `jobs.render_csv`. Считаются пары навыков ($k = 2$). Узлы ZooKeeper лежат под `/hhlake/<сценарий>/`.

- **Выборы.** Каждая копия создаёт в `election` эфемерный последовательный узел с именем `<uuid>-n_<номер>`. Лидер — владелец наименьшего 10-значного номера, запасной следит за узлом прямо перед собой. Уникальный префикс нужен на случай обрыва связи во время `create`: клиент не знает, создан ли узел, и сначала ищет свой узел по префиксу, а не создаёт второй. «Узел удалён» пишется только на событие `DELETED`. На прочих событиях (обрыв, переподключение) запасной просто перечитывает список. Если сессия запасного истекла и его узла больше нет, он встаёт в очередь заново.
- **Эпоха.** Победитель выборов берёт эпоху одной транзакцией: `check` своего узла выборов с `version=0` и `set` узла `epoch`. Номер новой версии `epoch` и есть номер эпохи. Если своего узла к этому моменту уже нет (сессия истекла, пока процесс стоял), транзакция не проходит, и координатор возвращается к выборам. Так эпоху нельзя получить, не будучи лидером в момент получения.
- **Чекпоинт.** После каждой map-задачи создаётся постоянный узел `done/map-XXX` с ключами её частей в S3 (`tmp/mr/zk-<сценарий>-e<эпоха>/...`) и счётчиками записей. Новый лидер берёт в работу только недостающие задачи, а reduce читает части по ключам из чекпоинта.
- **Публикация.** Файл кладётся в `results/zk/<сценарий>/epoch-<N>/k2.csv`, затем обновляется указатель `result` (заранее создан пустым, версия 0). Без защиты — обычный `set`. С `FENCING=1` — транзакция `check(epoch, version=моя_эпоха)` + `set_data(result)`. Перед публикацией лидер специально не проверяет, что он всё ещё лидер.
- **Сценарий** (`hhlake.zk.scenario`) запускает две копии `hhlake.zk.coord` по 1 воркеру. Когда в чекпоинте 13 из 26 задач, он шлёт `SIGSTOP` всей группе процессов лидера (координатор и воркеры пула), ждёт не меньше 40 с и до завершения запасного, затем шлёт `SIGCONT`. Если сценарий прервать, в `finally` он будит и завершает оставшиеся копии, чтобы они не висели остановленными. Искусственных задержек в расчёте нет.

Каждая копия пишет журнал с миллисекундами в `logs/<сценарий>/coord-{A,B}.log`, сценарий — в `scenario.log`. Полный вывод сценария лежит в `hw4/outputs/stage3_nofence.txt` и `stage4_fence.txt`. Состояние узлов я снимал через `zkCli` скриптом `zookeeper/state.sh`, оно в `stage3_zkcli.txt` и `stage4_zkcli.txt`.

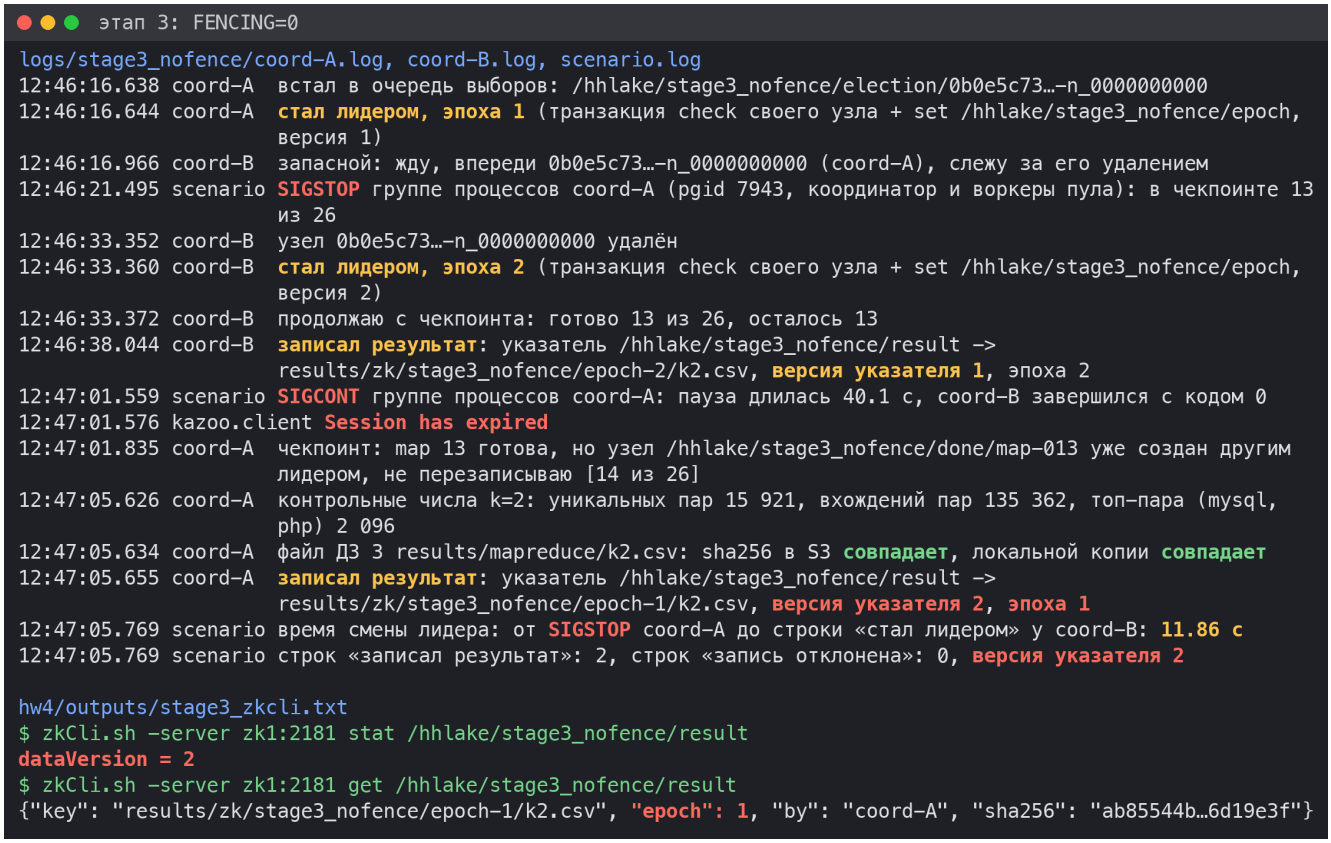


Рис. 3. Этап 3: две записи результата, указатель версии 2 от лидера эпохи 1 (фрагмент журналов `logs/stage3_nofence/*.log` и вывода `hw4/outputs/stage3_zkcli.txt`, строки отобраны, uuid и sha256 сокращены)

4 Этап 3: приостановка лидера без защиты

Таблица 2. Хронология `stage3_nofence` (`logs/stage3_nofence/`)

Время	Кто	Событие
12:46:16.644	coord-A	стал лидером, эпоха 1
12:46:21.495	сценарий	`SIGSTOP` coord-A, в чекпоинте 13 из 26
12:46:33.360	coord-B	стал лидером, эпоха 2
12:46:33.372	coord-B	продолжаю с чекпоинта: готово 13 из 26
12:46:38.044	coord-B	**записал результат**, версия указателя 1, эпоха 2
12:47:01.559	сценарий	`SIGCONT` coord-A, пауза 40,1 с, coord-B уже завершился
12:47:01.576	coord-A	`Session has expired`, затем новая сессия, расчёт продолжается
12:47:05.655	coord-A	**записал результат**, версия указателя 2, эпоха 1

Смена лидера: от `SIGSTOP` до строки «стал лидером» у запасного прошло **11,86 с** (по неокруглённому времени `SIGSTOP`). Из них 10 с — тайм-аут сессии (по `cons` согласованный тайм-аут 10 с). Лидер общался с ZooKeeper прямо перед остановкой, потому что только что записал чекпоинт. Остальные почти 2 с уходят на то, что сервер проверяет истечение сессий не непрерывно, а по границам `tickTime` = 2 с.

Запасной продолжил с чекпоинта и пересчитал только задачи 13–25. Обе копии сошлись с ДЗ 3: 15 921 уникальная пара, 135 362 вхождения, топ-пара (mysql, php) 2 096, 229 388 записей, 5 382 вакансии варианта. Хеш sha256 итогового CSV (`ab85544b…6d19e3f`) совпадает с `results/mapreduce/k2.csv` и в S3, и в локальной копии.

После `SIGCONT` координатор coord-A не знает, что он больше не лидер. Он досчитал свои задачи 13–25, но узлы чекпоинта для них уже создал coord-B, и coord-A их не перезаписал. Поэтому

его reduce прочитал тот же чекпоинт, что и reduce у coord-B: части 0–12 от coord-A и 13–25 от coord-B, а пересчитанные «зомби» части в итог не попали. Файлы совпали именно поэтому. Опасно другое: указатель переписал процесс устаревшей эпохи. В журналах **две** строки «записал результат», у указателя `dataVersion = 2`, и он ведёт на файл эпохи 1 от coord-A. Если бы за время паузы кто-то опубликовал более свежий результат, например следующий запуск на новых данных, проснувшийся «зомби» молча затёр бы его. Строки журналов и состояние указателя — на рис. 3.

5 Этап 4: то же с FENCING=1

Сценарий тот же, запуск `FENCING=1 uv run python -m hhlake.zk.scenario`. Лидер сменился за **12,15 с**. В 12:47:44.289 coord-B (эпоха 2) записал результат, указатель получил версию 1. В 12:48:07.240 сценарий разбудил coord-A. Тот досчитал, сошёлся с ДЗ 3 и в 12:48:11.490 пришёл с транзакцией `check(epoch, version=1)`, а версия `epoch` уже 2. Транзакция отклонена целиком с `BadVersionError`, и coord-A удалил свой неопубликованный файл `epoch-1/k2.csv`. Строка «записал результат» одна. Команда `stat` для указателя показывает `dataVersion = 1`, а `get` — ключ `epoch-2/k2.csv` от coord-B. В `results/zk/stage4_fence/` лежит один файл. Контрольные числа и sha256 у обеих копий снова совпали с ДЗ 3 (рис. 4).

этап 4: FENCING=1

```
logs/stage4_fence/coord-A.log, coord-B.log, scenario.log
12:47:22.456 coord-A  встал в очередь выборов: /hhlake/stage4_fence/election/1c8f37c2…-n_0000000000
12:47:22.460 coord-A  стал лидером, эпоха 1 (транзакция check своего узла + set /hhlake/stage4_fence/epoch,
                      версия 1)
12:47:22.784 coord-B  запасной: жду, впереди 1c8f37c2…-n_0000000000 (coord-A), слежу за его удалением
12:47:27.210 scenario SIGSTOP группе процессов coord-A (pgid 9907, координатор и воркеры пула): в чекпоинте 13
                      из 26
12:47:39.357 coord-B  узел 1c8f37c2…-n_0000000000 удалён
12:47:39.362 coord-B  стал лидером, эпоха 2 (транзакция check своего узла + set /hhlake/stage4_fence/epoch,
                      версия 2)
12:47:39.371 coord-B  продолжаю с чекпоинта: готово 13 из 26, осталось 13
12:47:44.289 coord-B  записал результат: указатель /hhlake/stage4_fence/result ->
                      results/zk/stage4_fence/epoch-2/k2.csv, версия указателя 1, эпоха 2
12:48:07.240 scenario SIGCONT группе процессов coord-A: пауза длилась 40.0 с, coord-B завершился с кодом 0
12:48:07.254 kazoo.client Session has expired
12:48:07.571 coord-A  чекпоинт: map 13 готова, но узел /hhlake/stage4_fence/done/map-013 уже создан другим
                      лидером, не перезаписываю [14 из 26]
12:48:11.457 coord-A  контрольные числа k=2: уникальных пар 15 921, вхождений пар 135 362, топ-пара (mysql,
                      php) 2 096
12:48:11.465 coord-A  файл ДЗ 3 results/mapreduce/k2.csv: sha256 в S3 совпадает, локальной копии совпадает
12:48:11.490 coord-A  запись отклонена: BadVersionError (check /hhlake/stage4_fence/epoch version=1, а текущая
                      эпоха 2)
12:48:11.505 coord-A  удалил свой неопубликованный файл s3://hh-lake/results/zk/stage4_fence/epoch-1/k2.csv
12:48:11.609 scenario время смены лидера: от SIGSTOP coord-A до строки «стал лидером» у coord-B: 12.15 с
12:48:11.609 scenario строк «записал результат»: 1, строк «запись отклонена»: 1, версия указателя 1

hw4/outputs/stage4_zkcli.txt
$ zkCli.sh -server zk1:2181 stat /hhlake/stage4_fence/result
dataVersion = 1
$ zkCli.sh -server zk1:2181 get /hhlake/stage4_fence/result
{"key": "results/zk/stage4_fence/epoch-2/k2.csv", "epoch": 2, "by": "coord-B", "sha256": "ab85544b…6d19e3f"}
```

Рис. 4. Этап 4: запись старого лидера отклонена, указатель версии 1 от запасного (фрагмент журналов `logs/stage4_fence/*.log` и вывода `hw4/outputs/stage4_zkcli.txt`, строки отобраны, uuid и sha256 сокращены)

6 Ответы на вопросы

Почему серверов три, а не два и не четыре. ZooKeeper работает, пока живо строгое большинство серверов: так два кворума всегда пересекаются, и у ансамбля не может появиться двух лидеров. Для двух серверов большинство — оба. Значит, отказ любого из них останавливает систему: два сервера переживают ноль отказов и надёжны хуже одного, ведь сломаться может любой из двух. Три сервера переживают один отказ. Я это проверил: без одного ансамбль работал, без двух — нет. Четыре сервера тоже переживают только один отказ, потому что большинство из четырёх —

три. Четвёртая машина запаса не даёт, зато каждая запись ждёт подтверждения от трёх серверов, а не от двух. Поэтому ансамбль делают нечётным: 3, 5, 7.

Почему эпоху проверяет получатель записи, а не ZooKeeper. Здесь ZooKeeper выступает службой сессий и выборов. Он замечает, что сессия старого лидера истекла, удаляет его узел и даёт выбрать нового, но остановить замороженный процесс не может. Тот проснётся когда угодно и продолжит с того места, где замер. Перепроверка «я ещё лидер?» перед записью тоже не спасает: пауза может случиться между проверкой и самой записью. Поэтому вместе с записью едет номер эпохи — токен, то есть число, которое растёт при каждой смене лидера. Сравнить его с последним виденным может только тот, кто принимает запись, и только атомарно с самой записью. В моей работе получатель записи — узел `result` в том же ZooKeeper, и сервер проверяет эпоху транзакцией `check(epoch)` + `set(result)`. Это роль получателя записи, а не службы выборов: так же условие проверяла бы любая база. Файл в S3 эпоху не проверяет, и в этапе 4 «зомби» свой файл положил (S3 его принял), а после отказа указателя сам удалил. Вреда от этого нет: у каждой эпохи свой ключ `epoch-N`, а результатом считается только то, на что ведёт указатель. Если бы результат писался в общий ключ S3 или в таблицу БД, токен должен был бы проверять сам S3 (условный PUT) или БД (`UPDATE ... WHERE epoch <= :token`).

7 Итог

Ансамбль из трёх серверов ZooKeeper 3.9.6 работает без одного сервера и перестаёт обслуживать запросы без двух. Эфемерный узел живёт, пока жива сессия его владельца. Запасной координатор стал лидером через 11,86 и 12,15 с после остановки основного и продолжил расчёт с чекпоинта. Результат в обоих запусках совпал с ДЗ 3 до байта. Без защиты проснувшийся «зомби» перезаписал указатель (две записи, версия 2). С проверкой эпохи на стороне получателя его запись отклонена с `BadVersionError`, и указатель остался от запасного (версия 1).